

UNIT-3

6. a) Explain the Classification of Frequent Pattern Mining

Frequent Pattern Mining is the process of finding recurring relationships or patterns in large datasets, such as items bought together in a transaction.

It can be classified into the following types:

1. Frequent Itemset Mining:

- Identifies sets of items that frequently occur together.
- Example: {milk, bread} appears in 40% of transactions.
- Algorithms: Apriori, FP-Growth.

2. Frequent Subsequence Mining:

- Finds frequently occurring sequences or ordered events.
- Example: A user visits Homepage → Product Page → Checkout frequently.
- Used in web log analysis, clickstream data.

3. Frequent Substructure Mining:

- Extracts frequently occurring structured patterns such as graphs, trees, or subgraphs.
- Used in chemical compound analysis, social networks.

4. Maximal and Closed Frequent Pattern Mining:

- **Closed Patterns:** Patterns that have no super-pattern with the same support.
- **Maximal Patterns:** Patterns that have no frequent super-patterns.
- Helps in reducing the number of patterns without losing essential information.

5. Constraint-Based Frequent Pattern Mining:

- Finds patterns that meet specific user-defined constraints (e.g., min value, time limit).
- Makes pattern mining more focused and efficient.

6. b) Explain Any Three Pruning Strategies of Mining Closed Frequent Itemsets

Closed frequent itemset mining can produce a large number of patterns. **Pruning strategies** help eliminate unneeded candidates to improve performance.

1. Support-Based Pruning:

- If the support of an itemset is below the **minimum support threshold**, it and its supersets are pruned.
 - Helps reduce the number of candidates early in the process.
-

2. Subset Checking for Closure Property:

- If an itemset has the same support as its immediate supersets, it is **not a closed itemset** and can be pruned.
 - Only itemsets with no supersets having the same support are kept.
-

3. Itemset Extension Pruning:

- During pattern growth (e.g., in FP-Growth), certain extensions that cannot lead to frequent closed itemsets are avoided.
 - Example: If item Y cannot co-occur frequently with item X, we skip combining them.
-

Here's a detailed and easy-to-understand answer for **Question 7 (a & b)**, as expected in a 12-mark exam (typically split as 6 + 6 marks):

7. a) Explain Apriori Algorithm

The **Apriori algorithm** is a classic algorithm used in **frequent itemset mining** and **association rule learning** over transactional databases.

Key Idea:

- It uses the **Apriori property**, which states:

"All non-empty subsets of a frequent itemset must also be frequent."

Steps in Apriori Algorithm:

1. **Generate Frequent 1-Itemsets (L1):**
 - Scan the database to find items that meet the minimum support.
 2. **Generate Candidate k-Itemsets (Ck):**
 - Use L_{k-1} to generate C_k (e.g., from L₂, generate C₃).
 - Candidate itemsets are formed by self-joining and pruning.
 3. **Prune:**
 - Eliminate candidates in C_k that have infrequent subsets.
 4. **Scan and Count Support:**
 - Scan the database to count the support of candidates in C_k.
 - Those that meet minimum support become L_k (frequent k-itemsets).
 5. **Repeat:**
 - Repeat steps 2–4 until no more frequent itemsets can be found.
-

Example:

- **Transaction DB:**
 - T1: {A, B, C}
 - T2: {A, C}
 - T3: {A, B}
 - T4: {B, C}
 - **Min Support = 2**
 - L1: {A}, {B}, {C}
 - L2: {A, B}, {A, C}, {B, C}
 - L3: No candidates meet support → Stop
-

7. b) What is Constraint-Based Mining? Explain the Constraints Included

Definition:

Constraint-Based Mining is a type of data mining where the user **specifies constraints** to focus the mining process and **filter out uninteresting patterns**.

It improves **efficiency**, **accuracy**, and **relevance** of the results.

Types of Constraints in Constraint-Based Mining:

1. Knowledge-Type Constraints:

- Specify the kind of patterns to be found.
 - Example: Only mine **association rules** or **sequential patterns**.
-

2. Data Constraints:

- Apply constraints on **data values or ranges**.
 - Example: Only include transactions from "2023" or items with price > ₹100.
-

3. Dimension Constraints:

- Used in **multidimensional mining**, to restrict mining to certain dimensions.
 - Example: Only mine sales patterns by "region" and not "time".
-

4. Interestingness Constraints:

- Filters results based on **support, confidence, lift**, etc.
 - Example: Only show rules with **confidence > 70%**.
-

5. Rule Form Constraints:

- Restrict the form or structure of rules.
 - Example: Only allow rules of the form $A \rightarrow B$ where A has exactly 2 items.
-

Benefits of Constraint-Based Mining:

- Reduces computational cost
 - Avoids mining irrelevant patterns
 - Focuses on patterns that are **meaningful to users**
-

UNIT-4

Here is a clear and complete answer for **Question 8 (a, b, c)** suitable for **12 marks** (typically 4+4+4 marks format):

8. a) Explain Any Four Criteria Used for Comparing Classification and Prediction Methods

When evaluating classification and prediction techniques, several criteria are used. Here are four key ones:

1. Accuracy:

- Measures the percentage of correctly predicted instances out of total instances.
 - Higher accuracy indicates a better model.
 - Example: A model that correctly classifies 90 out of 100 instances has 90% accuracy.
-

2. Speed (Training and Testing Time):

- Refers to the time taken to build (train) the model and the time to classify new data (testing).
 - Some algorithms are faster than others (e.g., Decision Trees are faster than SVMs for training).
-

3. Interpretability:

- Indicates how easy it is to understand the model's structure and decisions.
 - Example: Decision Trees are highly interpretable, while Neural Networks are considered black-box models.
-

4. Scalability:

- Refers to the model's ability to handle large volumes of data efficiently.
 - Some models perform well with small datasets but struggle with very large datasets.
-

8. b) Explain IF-THEN Rules for Classification

IF-THEN rules are a simple and interpretable form of classification.

Structure:

IF condition THEN class

- The condition (antecedent) specifies a test on one or more attributes.

- The THEN part (consequent) assigns a class label.
-

Example:

IF age < 25 AND income = high THEN class = Yes

Advantages:

- Easy to understand and implement.
 - Can be derived from decision trees or other models.
 - Flexible and can be easily updated or combined.
-

Rule Extraction:

- Algorithms like **RIPPER**, **C4.5**, or **PART** are used to extract rules from data.
-

8. c) Explain DBSCAN and OPTICS

Both DBSCAN and OPTICS are **density-based clustering algorithms**, suitable for discovering clusters of arbitrary shape and handling noise.

1. DBSCAN (Density-Based Spatial Clustering of Applications with Noise):

Key Concepts:

- Clusters are formed based on **density** (minimum number of points within a given radius).
 - Two parameters:
 - **Eps (ϵ):** Neighborhood radius
 - **MinPts:** Minimum number of points in the ϵ -neighborhood
-

Working:

- Core points: Points with at least MinPts in ϵ -radius.
 - Border points: Less than MinPts but near a core point.
 - Noise: Points not in any cluster.
-

Advantages:

- Handles noise and outliers well.
 - Can find clusters of arbitrary shape.
-

Limitations:

- Sensitive to parameter selection (ϵ and MinPts).
-

2. OPTICS (Ordering Points To Identify the Clustering Structure):

Extension of DBSCAN that handles **varying density clusters**.

Key Features:

- Produces an ordering of points based on **reachability distance**.
 - Instead of explicitly forming clusters, it outputs a **reachability plot** from which clusters can be derived.
-

Advantages:

- More flexible than DBSCAN.
 - Detects nested clusters and clusters with different densities.
-

Use Case Comparison:

Algorithm	Handles Noise	Varying Density	Output
DBSCAN	Yes	No	Cluster labels
OPTICS	Yes	Yes	Reachability plot

Here's a detailed and easy-to-understand answer for **Question 9 (a, b, c)** — suitable for 12 marks (typically 4 + 4 + 4 marks).

9. a) Explain Any Four Requirements of Clustering in Data Mining

Clustering is a key technique in data mining. Good clustering methods should satisfy the following requirements:

1. Scalability:

- The algorithm should be able to handle large volumes of data efficiently.
 - Example: Should work well with datasets having millions of records.
-

2. Ability to Deal with Different Types of Attributes:

- Should support numerical, categorical, binary, or mixed-type attributes.
 - Example: Age (numeric), Gender (categorical), etc.
-

3. Ability to Discover Arbitrary Shaped Clusters:

- Not all clusters are spherical; some may be elongated, nested, etc.
 - The algorithm should be flexible to detect clusters of any shape.
-

4. Minimal Requirements for Domain Knowledge:

- The algorithm should not require users to specify too many parameters or deep domain expertise.
 - Example: Should ideally work even if the number of clusters is not known in advance.
-

9. b) Explain K-Means Partition Algorithm

K-Means is a popular **partitioning clustering algorithm** that divides the data into **K non-overlapping clusters**.

Steps:

1. **Initialization:**
Select K initial cluster centers (centroids) randomly.
2. **Assignment Step:**
Assign each data point to the nearest cluster centroid using **Euclidean distance**.
3. **Update Step:**
Recalculate the centroid of each cluster based on the current members.

4. **Repeat:**

Repeat steps 2 and 3 until centroids do not change (convergence).

Advantages:

- Simple and fast.
- Works well for spherical, well-separated clusters.

Limitations:

- Sensitive to initial centroids.
 - Not suitable for clusters with non-spherical shapes or different densities.
-

9. c) Explain Four Cases of Cost Function for K-Medoids Clustering

K-Medoids is similar to K-means but uses **medoids** (actual data points) instead of centroids. It minimizes a **cost function**, often called **total dissimilarity**.

Here are four cases for evaluating the **cost function**:

1. Inserting a Point into a Cluster:

- When a point is added to a cluster, the cost increases based on how different the point is from the medoid.
 - Cost $\uparrow$ if the point is far from the current medoid.
-

2. Swapping the Medoid with a Non-Medoid:

- A new medoid is chosen and the cost is recalculated.
 - If the new medoid reduces the total distance to other points, Cost $\downarrow$.
-

3. Removing a Point from a Cluster:

- If a non-medoid point is removed, the cost remains the same.
 - If the medoid is removed, the cost increases until a new suitable medoid is selected.
-

4. Merging or Splitting Clusters:

- **Merging** clusters can increase cost if the combined points are not close to one medoid.
- **Splitting** may reduce cost by assigning points to a more appropriate medoid.

K-Medoids Cost Function:

$$\text{Total Cost} = \sum_{i=1}^k \sum_{x \in C_i} \text{distance}(x, \text{medoid}_i)$$
